Guía de estudio — las 29 rúbricas en el proyecto Pedro

Para qué sirve: aprender cada rúbrica en pocos minutos. Cada ficha responde cuatro preguntas: qué es el concepto, en qué consiste en Pedro, dónde está en el proyecto y cómo verlo funcionar.

Cómo usarla: léela en el orden de las fases. Cada fase construye sobre la anterior. Para el detalle y las evidencias de cada rúbrica, abre docs/respostas_rubricas.md en la sección con el mismo código.

Nota de idioma: esta guía está en español para estudiar. El proyecto está en portugués, así que los nombres de archivos, funciones y notebooks aparecen tal como están en el código.

1. El proyecto en un minuto

Pedro es un asistente de compras de piezas de electrodomésticos. Todos los datos son ficticios. El usuario pide una pieza y una cantidad; Pedro compara los proveedores y recomienda uno.

- **El núcleo es comparar proveedores.** La función comparar en src/pedro/business.py descarta ofertas por stock, pedido mínimo, plazo o presupuesto y ordena las que quedan.
- **La idea que ordena todo:** el modelo decide qué responder; los datos los pone el sistema. Precios, totales y citas nunca salen de la "memoria" del modelo.
- **Nada se publica sin verificación.** Cada respuesta pasa por una comprobación exacta y por un modelo de NLI antes de mostrarse.
- **Todo el dinero es entero, en centavos.** Nunca float.

2. Mapa del proyecto

Carpeta o archivo	Qué contiene
app.py	La interfaz Streamlit (solo el chat)
src/pedro/	Toda la lógica reutilizable
scripts/demonstrar_*.py	Demostraciones con el modelo real; guardan un JSON en docs/rastros/
notebooks/	Cinco notebooks didácticos, uno por fase, con dobles de prueba
tests/	188 pruebas automáticas
evals/	Casos de evaluación y el evaluador de LangSmith
knowledge/	Corpus ficticio de políticas y el índice FAISS guardado
datasets/	Catálogo SQLite sintético: 18 categorías, 4.320 piezas, 12.960 ofertas
docs/	Respuestas a las rúbricas, informe, matriz, evidencias y trazas

3. Las cinco fases

Fase	Rúbricas	Idea	Notebook
1	R01–R06	Una cadena que extrae la solicitud, y la arquitectura del agente	01_chain_mvp
2	R07–R12	La cadena pasa a ser un agente con herramientas (ReAct)	02_react_ferramentas
3	R13–R18	El agente recuerda: memoria y conversación	03_memoria_chatbot
4	R19–R23	El agente consulta datos: SQL, APIs y RAG	04_rag_text_to_query
5	R24–R29	Todo junto, verificado, en la interfaz y evaluado	05_integrador_avaliacao

4. Fase 1 — Chain ejecutable y arquitectura

R01 — Anatomía, tipo de agente y madurez

- **El concepto:** un agente tiene partes: motor cognitivo (el LLM), herramientas, memoria, base de conocimiento y orquestador. Puede ser reactivo (reglas fijas), deliberativo (razona) o híbrido.
- **En Pedro:** es un agente híbrido. La capa reactiva son reglas que resuelven lo simple sin llamar al modelo. La capa deliberativa es el LLM, que elige herramientas.
- **Dónde está:** src/pedro/anatomia.py (descrever_anatomia, inventario_langchain); las dos capas viven en src/pedro/agent.py (criar_fluxo).
- **Verlo funcionar:** notebook 01, secciones R01; tests/test_anatomia.py.
- **Para recordar:** híbrido = reglas para lo previsible, modelo para el resto.

R02 — Planificación y memoria aumentada

- **El concepto:** planificar es dividir una petición compleja en pasos. Memoria aumentada es guardar fuera del modelo lo que necesita recordar.
- **En Pedro:** cuando un mensaje tiene dos o más intenciones, el modelo escribe un plan validado (PlanoTurno), con una petición por intención. Cada petición se resuelve por el camino normal, con su propia verificación.
- **Dónde está:** src/pedro/planner.py (intencoes_detectadas, planejar_turno, validar_plano); memoria en src/pedro/memory.py.
- **Verlo funcionar:** scripts/demonstrar_r02.py; traza docs/rastros/r02_planejamento_llm.json.
- **Para recordar:** el plan no puede contener datos que el usuario no escribió.

R03 — Plantillas de prompt reutilizables

- **El concepto:** PromptTemplate y ChatPromptTemplate son prompts con variables, escritos una vez y usados muchas veces.
- **En Pedro:** doce plantillas con nombre, por ejemplo MODELO_EXTRACAO y MODELO_AGENTE.
- **Dónde está:** src/pedro/prompts.py.
- **Verlo funcionar:** notebook 01, sección R03; tests/test_prompts.py.
- **Para recordar:** el prompt es código: tiene nombre, variables y prueba.

R04 — Output parser con salida JSON

- **El concepto:** un parser convierte el texto del modelo en datos estructurados y valida el formato.
- **En Pedro:** PydanticOutputParser con los contratos de contracts.py. Hay una comprobación extra de JSON completo, porque un parser tolerante puede aceptar un JSON cortado y perder datos en silencio.
- **Dónde está:** src/pedro/chains.py (_json_completo, criar_lcel_robusta); contratos en src/pedro/contracts.py (Extracao, Solicitacao).
- **Verlo funcionar:** notebook 01, secciones R04; scripts/demonstrar_fase1.py; tests/test_saida_estruturada.py.
- **Para recordar:** validar la forma no basta; hay que validar que la salida está entera.

R05 — invoke, batch, stream y @chain

- **El concepto:** todo Runnable tiene tres formas de ejecución: invoke (una entrada), batch (varias en paralelo) y stream (trozos según llegan). @chain convierte una función Python en Runnable.
- **En Pedro:** la misma cadena de extracción se ejecuta de las tres formas y también está escrita con @chain.
- **Dónde está:** src/pedro/chains.py (criar_chain, criar_chain_robusta, criar_lcel_parcial).
- **Verlo funcionar:** notebook 01, secciones R05; scripts/demonstrar_fase1.py (mide los tiempos).
- **Para recordar:** batch fue unas tres veces más rápido que tres invoke en serie.

R06 — LCEL: declarativo frente a imperativo

- **El concepto:** LCEL compone pasos con el operador | (declarativo). La alternativa es escribir el flujo como código normal (imperativo).
- **En Pedro:** la misma cadena con reparación existe en los dos estilos, y un laboratorio compara lo que cada uno deja ver en la traza.
- **Dónde está:** src/pedro/chains.py (criar_lcel, criar_lcel_robusta, criar_chain_robusta) y src/pedro/laboratorio.py (comparar_estilos, ensaiar).
- **Verlo funcionar:** notebook 01, sección R06.
- **Para recordar:** LCEL muestra cada paso en la traza; el imperativo es más fácil de leer cuando hay ramificaciones.

5. Fase 2 — Agente con herramientas y ReAct

R07 — Comparar orquestadores

- **El concepto:** el orquestador coordina modelo, herramientas y memoria. LangChain clásico y LangGraph son dos opciones.
- **En Pedro:** el mismo agente de dos herramientas se construyó con los dos y se comparó. El producto usa LangChain.
- **Dónde está:** src/pedro/orquestradores.py (agente_langchain, agente_langgraph); texto en docs/comparacoes.md.
- **Verlo funcionar:** scripts/demonstrar_r07.py; traza docs/rastros/r07_orquestradores_llm.json.
- **Para recordar:** la elección se justifica con una comparación ejecutada, no con una opinión.

R08 — ReAct con initialize_agent

- **El concepto:** ReAct es el ciclo Thought, Action, Observation: el modelo piensa, usa una herramienta, lee el resultado y repite.
- **En Pedro:** un agente ReAct clásico con initialize_agent, dos herramientas y memoria de conversación.
- **Dónde está:** src/pedro/legacy.py (criar_react_textual).
- **Verlo funcionar:** notebook 02, sección R08; scripts/demonstrar_r08.py; tests/test_legacy.py.
- **Para recordar:** con la memoria en el prompt, el agente entiende "¿y de esa pieza?"; sin ella, vuelve a pedir el ID.

R09 — Chains secuenciales, paralelas y RouterChain

- **El concepto:** las cadenas pueden ejecutarse en secuencia, en paralelo (RunnableParallel) o ser elegidas por un enrutador.
- **En Pedro:** una composición paralela y dos enrutadores: uno por reglas (RouterCompras) y otro en el que el LLM elige el destino.
- **Dónde está:** src/pedro/chains.py (composicao_paralela) y src/pedro/legacy.py (RouterCompras, criar_router_llm, executar_rota).
- **Verlo funcionar:** notebook 02, sección R09; scripts/demonstrar_r09.py.
- **Para recordar:** con las mismas preguntas, las reglas acertaron 2 de 7 rutas y el LLM 6 de 7.

R10 — API externa por function calling

- **El concepto:** function calling es que el modelo pida ejecutar una función con argumentos estructurados.
- **En Pedro:** la API pública ViaCEP es la herramienta consultar_cep. La ciudad publicada sale de la respuesta de la API, no del modelo.
- **Dónde está:** src/pedro/integrations.py (consultar_cep); herramienta en src/pedro/agent.py; anclaje en src/pedro/verification.py (ancorar_cep).

- **Verlo funcionar:** scripts/demonstrar_r10.py; tests/test_policy_cep.py.
- **Para recordar:** el modelo elige llamar; el dato publicado es el que devolvió la API.

R11 — PTCF, few-shot y chain-of-thought

- **El concepto:** PTCF es estructurar el prompt en Persona, Tarea, Contexto y Formato. Few-shot es dar ejemplos. Chain-of-thought es pedir el razonamiento antes de la respuesta.
- **En Pedro:** se midieron tres variantes del prompt de extracción con las mismas diez solicitudes.
- **Dónde está:** src/pedro/prompts.py (MODELO_EXTRACAO, MODELO_EXTRACAO_FEWSHOT, MODELO_EXTRACAO_COT, MODELO_AGENTE); src/pedro/chains.py (criar_variantes_de_prompt).
- **Verlo funcionar:** scripts/demonstrar_r11.py; traza docs/rastros/r11_prompts_llm.json.
- **Para recordar:** zero-shot acertó 6 de 10 solicitudes; few-shot y few-shot con CoT, 10 de 10.

R12 — Componentes de confiabilidad

- **El concepto:** parsers con manejo de errores, fallbacks, memoria, índice vectorial y callbacks para saber qué pasó.
- **En Pedro:** un laboratorio de salidas rotas muestra qué capa resuelve cada defecto. El callback Rastro registra herramientas, verificaciones y errores de cada turno.
- **Dónde está:** src/pedro/laboratorio.py (SAIDAS_QUEBRADAS, ensaiar); src/pedro/agent.py (clase Rastro).
- **Verlo funcionar:** notebook 01, sección R04; notebook 02, sección R12.
- **Para recordar:** sin registrar el motivo de cada rechazo, no se puede corregir el agente.

6. Fase 3 — Chatbot con memoria

R13 — Memoria de corto y largo plazo (CoALA)

- **El concepto:** la taxonomía CoALA separa memoria de trabajo (corto plazo), episódica (lo que pasó), semántica (hechos) y procedimental (cómo actuar).
- **En Pedro:** mensajes y borrador de compra (trabajo), episodios con embeddings (episódica), preferencias y políticas (semántica), prompts y reglas (procedimental).
- **Dónde está:** src/pedro/memory.py (mapa_memorias, Historico, salvar_rascunho, salvar_preferencia, salvar_episodio).
- **Verlo funcionar:** notebook 03, sección R13; scripts/demonstrar_fase3.py.
- **Para recordar:** cada tipo de memoria responde una pregunta distinta: qué estamos haciendo, qué pasó, qué es verdad, cómo actuar.

R14 — Ventana de contexto y búsqueda vectorial

- **El concepto:** el modelo solo ve una ventana de los últimos mensajes. Lo que quedó fuera se recupera por similitud de embeddings.
- **En Pedro:** ventana de 12 mensajes y recuperación de los tres episodios más parecidos, con FastEmbed.
- **Dónde está:** src/pedro/memory.py (JANELA_MENSAGENS, recuperar, FastEmbedder).
- **Verlo funcionar:** notebook 03; traza docs/rastros/fase3_memoria_llm.json.
- **Para recordar:** ventana para lo reciente, vector para lo antiguo.

R15 — Comparar herramientas de memoria

- **El concepto:** existen bibliotecas especializadas en memoria de agentes, como LangMem, Mem0 y Letta.
- **En Pedro:** la memoria propia se comparó con LangMem en una ejecución real; Mem0 y Letta quedan como comparación de arquitectura.
- **Dónde está:** src/pedro/orquestradores.py (agente_com_langmem); texto en docs/comparacoes.md.
- **Verlo funcionar:** scripts/demonstrar_r15.py; traza docs/rastros/r15_memoria_llm.json.
- **Para recordar:** en Pedro el sistema decide qué guardar; en LangMem lo decide el modelo.

R16 — ConversationBufferMemory

- **El concepto:** la memoria más simple: guardar toda la conversación y ponerla entera en el prompt.
- **En Pedro:** un chatbot clásico con ConversationChain, para mostrar que el buffer crece en cada turno.
- **Dónde está:** src/pedro/legacy.py (criar_chatbot_buffer, MODELO_CHATBOT).
- **Verlo funcionar:** notebook 03, sección R16.
- **Para recordar:** el buffer crece sin límite; por eso el producto usa ventana y búsqueda vectorial.

R17 — Intención y extracción de entidades

- **El concepto:** entender qué quiere el usuario (intención) y los datos que dio (entidades: pieza, cantidad, presupuesto, plazo).
- **En Pedro:** reglas para el ID de la pieza y un intérprete few-shot (MudancaSolicitacao) que decide si el mensaje es una compra y qué cambia.
- **Dónde está:** src/pedro/dialogue.py (interpretar_compra, interpretar_turno, criar_interprete).
- **Verlo funcionar:** scripts/demonstrar_r17.py; tests/test_dialogue.py.
- **Para recordar:** solo con reglas se acertaron 8 de 14 mensajes; con reglas y modelo, 14 de 14.

R18 — Contexto y descomposición en una conversación real

- **El concepto:** mantener el estado a lo largo de varios turnos y dividir mensajes con varias peticiones.
- **En Pedro:** una conversación real de ocho turnos cambia la cantidad, quita el plazo, guarda una preferencia, pregunta un CEP y una política, y vuelve a la compra sin perder el estado.
- **Dónde está:** src/pedro/dialogue.py (borrador por sesión) y src/pedro/planner.py; flujo en src/pedro/agent.py.
- **Verlo funcionar:** scripts/demonstrar_fase3.py; traza docs/rastros/fase3_memoria_llm.json.
- **Para recordar:** Pedro solo pregunta el dato que bloquea (la cantidad); presupuesto y plazo son filtros opcionales.

7. Fase 4 — SQL, APIs y RAG

R19 — Herramientas hardcoded y semánticas

- **El concepto:** una herramienta hardcoded ejecuta código fijo; una semántica depende de una búsqueda por significado.
- **En Pedro:** seis herramientas. comparar_ofertas es cálculo fijo; consultar_politicas busca por significado en el índice FAISS.
- **Dónde está:** src/pedro/agent.py (función ferramentas); papeles descritos en src/pedro/anatomia.py (PAPEIS).
- **Verlo funcionar:** tests/test_tools_flow.py; notebook 05.
- **Para recordar:** las cuentas y las reglas van en código fijo; el modelo solo elige cuándo llamar.

R20 — APIs internas y externas

- **El concepto:** una API interna es de la propia aplicación; una externa es de terceros y puede fallar.
- **En Pedro:** interna, el catálogo SQLite en modo solo lectura; externa, ViaCEP, con manejo de CEP inválido, inexistente y servicio caído.
- **Dónde está:** src/pedro/catalog.py y src/pedro/integrations.py.
- **Verlo funcionar:** scripts/demonstrar_r10.py; tests/test_rag_integrations.py.
- **Para recordar:** toda llamada externa necesita un plan para cuando falla.

R21 — Text-to-query

- **El concepto:** convertir una pregunta en lenguaje natural en una consulta a la base de datos.

- **En Pedro:** dos caminos. Con ID de pieza, un plan Pydantic se convierte en un SELECT parametrizado. Para preguntas abiertas, el LLM escribe el SQL, que pasa por una lista blanca de tablas y columnas, solo lectura, límite de filas y un intento de reparación.
- **Dónde está:** src/pedro/query.py (planejar, executar, validar_sql, consultar_sql, responder_sql).
- **Verlo funcionar:** notebook 04, sección R21; scripts/demonstrar_fase4.py; tests/test_text_to_sql.py.
- **Para recordar:** el modelo escribe la consulta; quien decide si se ejecuta es el motor.

R22 — RAG sobre un índice FAISS existente

- **El concepto:** RAG es recuperar fragmentos relevantes de documentos y responder a partir de ellos. Aquí el índice ya existe en disco: se carga, no se construye.
- **En Pedro:** el índice guardado en knowledge/politicas/ solo se abre después de comprobar el manifiesto (origen, modelo, dimensión, métrica y SHA-256), porque el archivo es pickle.
- **Dónde está:** src/pedro/rag.py (inspecionar_indice, carregar_indice, consultar).
- **Verlo funcionar:** notebook 04, sección R22; scripts/demonstrar_r22.py; tests/test_indice_existente.py.
- **Para recordar:** PEDRO_TRUST_POLICY_INDEX=true solo después de verificar la procedencia del índice.

R23 — Fallo real de recuperación, diagnosticado y corregido

- **El concepto:** la búsqueda vectorial puede no traer el fragmento correcto. Hay que medir el fallo, entender la causa y corregirlo.
- **En Pedro:** la pregunta coloquial «Posso receber em várias caixas?» no trae la regla de entrega parcial en el top-4. Reescrita como pregunta sobre envío parcial, la regla sube a la posición 1.
- **Dónde está:** src/pedro/policy_demo.py (reescrever_consulta_entrega); src/pedro/rag.py (diagnosticar_reescrita); scripts/diagnosticar_t24.py; texto en docs/t24_recuperacao.md.
- **Verlo funcionar:** notebook 04, sección T24; en el chat, el comando /t24.
- **Para recordar:** el usuario habla de "cajas", el documento habla de "entrega parcial". La reescritura acerca los dos vocabularios.

8. Fase 5 — Agente final verificado y evaluado

R24 — Verificación factual y coherencia

- **El concepto:** comprobar que la respuesta del modelo es fiel a las fuentes. NLI (inferencia en lenguaje natural) dice si un texto está respaldado por otro.
- **En Pedro:** dos puertas. La verificación exacta comprueba ganador, total, fuentes y dataset. El NLI comprueba si el texto está respaldado. Antes, el sistema ancla los datos: cambia las cifras por el resultado de la herramienta y las citas por el texto literal del fragmento.
- **Dónde está:** src/pedro/verification.py (verificar, ancorar_recomendacao, ancorar_citacoes, ancorar_cep, ancorar_catalogo) y src/pedro/nli.py (verificar_nli); texto en docs/citacoes_ancoradas.md.
- **Verlo funcionar:** tests/test_core.py, tests/test_citacoes_ancoradas.py; notebook 05.
- **Para recordar:** en diez experimentos no se publicó ninguna recomendación contraria a la regla.

R25 — Un agente con dos o más categorías de herramientas

- **El concepto:** un único agente que combina SQL, RAG y API.
- **En Pedro:** el mismo ejecutor usa consultar_catalogo (SQL), consultar_politicas (RAG) y consultar_cep (API), además de comparar_ofertas, buscar_produtos y salvar_preferencia.
- **Dónde está:** src/pedro/agent.py (ferramentas, criar_agente).
- **Verlo funcionar:** scripts/demonstrar_fase3.py (las tres categorías en la misma conversación).
- **Para recordar:** una conversación, un agente, tres tipos de fuente.

R26 — create_openai_tools_agent y AgentExecutor

- **El concepto:** create_openai_tools_agent crea el agente que usa function calling; AgentExecutor ejecuta el ciclo de llamar herramientas hasta tener la respuesta.
- **En Pedro:** seis herramientas, como máximo seis iteraciones y una única pasada de corrección cuando la verificación rechaza la primera propuesta.
- **Dónde está:** src/pedro/agent.py (criar_agente, criar_fluxo).
- **Verlo funcionar:** tests/test_tools_flow.py; scripts/demonstrar_fase3.py.
- **Para recordar:** los límites de iteraciones y de corrección existen para que la autonomía esté controlada.

R27 — RunnableWithMessageHistory

- **El concepto:** envuelve una cadena y se encarga de leer y guardar el historial de cada sesión.
- **En Pedro:** el historial está en SQLite, por usuario y sesión. Solo se guarda la respuesta pública, nunca el razonamiento interno ni las propuestas rechazadas.
- **Dónde está:** src/pedro/agent.py (criar_fluxo, mensagem_publica) y src/pedro/memory.py (Historico).
- **Verlo funcionar:** tests/test_tools_flow.py; reabrir una conversación en la interfaz.
- **Para recordar:** lo que el usuario no vio no entra en el historial.

R28 — Integración en Streamlit

- **El concepto:** conectar el agente completo a una interfaz.
- **En Pedro:** un chat que muestra los pasos del agente, cuatro tarjetas (proveedor, total, entrega y origen), el consejo o aviso de precio y la tabla de ofertas. La interfaz dibuja a partir de los datos verificados que viajan con el mensaje, sin releer el texto.
- **Dónde está:** app.py; tema en .streamlit/config.toml.
- **Verlo funcionar:** ./project.sh start y abrir http://localhost:8501; scripts/demonstrar_r28.py; capturas en docs/capturas/.
- **Para recordar:** app.py solo dibuja; toda la lógica está en src/pedro/.

R29 — Evaluación con LangSmith

- **El concepto:** medir el agente con un conjunto de casos y evaluadores, tanto la calidad de la respuesta como la trayectoria (qué herramientas usó).
- **En Pedro:** 30 casos derivados y seis evaluadores: ganador, importe, contenido, trayectoria, herramienta decisiva y primera propuesta.
- **Dónde está:** evals/avaliar_fluxo.py; casos en evals/casos_derivados.json; generador en scripts/criar_casos_eval.py.
- **Verlo funcionar:** traza docs/rastros/r29_langsmith_gpt-4.1-mini_portugues.json; tests/test_evals.py.
- **Para recordar:** ganador 14 de 14, importe 14 de 14, contenido 16 de 16; trayectoria completa 15 de 30 en el experimento vigente (26 en el anterior): el agente llega al mismo resultado por un camino más corto que el de la referencia. Nunca simular métricas.

Fuera de las rúbricas — Guarda de entrada y modelo local

- **El concepto:** una guarda de entrada (guardrail) decide si un mensaje puede entrar al agente. Un modelo local es un modelo de lenguaje que se ejecuta en la propia máquina.
- **En Pedro:** reglas fijas rechazan lo que está claro (intento de cambiar las instrucciones, tema ajeno) y un clasificador con el modelo decide el resto. El mensaje rechazado recibe un texto fijo y no llega a las herramientas. El modelo de conversación se elige en el .env: OpenAI o un servidor local, como Ollama. Si el agente no redacta una respuesta válida, el sistema publica su propio cálculo, verificado.
- **Dónde está:** src/pedro/guardrails.py, src/pedro/modelos.py y resolver, en src/pedro/agent.py.
- **Verlo funcionar:** scripts/demonstrar_guardrails.py, scripts/demonstrar_modelo_local.py; tests/test_guardrails.py; en el chat, «Qual o melhor celular de 2026?».

- **Para recordar:** 28 de 28 mensajes bien clasificados (21 por las reglas, 7 por el modelo). Con un modelo local en CPU, cada turno con herramienta tardó minutos.

9. Resumen de una página

Rúbrica	En una frase	Archivo principal
R01	Agente híbrido: reglas y modelo	src/pedro/anatomia.py
R02	Plan validado para mensajes con varias intenciones	src/pedro/planner.py
R03	Doce plantillas de prompt con nombre	src/pedro/prompts.py
R04	Parser Pydantic y comprobación de JSON completo	src/pedro/chains.py
R05	invoke, batch, stream y @chain	src/pedro/chains.py
R06	La misma cadena en LCEL y en código imperativo	src/pedro/laboratorio.py
R07	LangChain comparado con LangGraph	src/pedro/orquestradores.py
R08	Agente ReAct clásico	src/pedro/legacy.py
R09	Paralelo y enrutadores	src/pedro/legacy.py
R10	ViaCEP como herramienta	src/pedro/integrations.py
R11	PTCF, few-shot y CoT medidos	src/pedro/prompts.py
R12	Laboratorio de fallos y callback Rastro	src/pedro/laboratorio.py
R13	Cuatro memorias de CoALA	src/pedro/memory.py
R14	Ventana de 12 mensajes y episodios por vector	src/pedro/memory.py
R15	Memoria propia comparada con LangMem	src/pedro/orquestradores.py
R16	Chatbot con buffer completo	src/pedro/legacy.py
R17	Reglas e intérprete few-shot	src/pedro/dialogue.py
R18	Ocho turnos reales sin perder el estado	src/pedro/dialogue.py
R19	Herramienta fija y herramienta semántica	src/pedro/agent.py
R20	SQLite interno y ViaCEP externo	src/pedro/catalog.py
R21	Plan a SELECT y text-to-SQL con lista blanca	src/pedro/query.py
R22	Índice FAISS guardado, abierto solo con manifiesto comprobado	src/pedro/rag.py
R23	Fallo de recuperación medido y corregido con reescritura	src/pedro/policy_demo.py
R24	Anclaje, verificación exacta y NLI	src/pedro/verification.py
R25	SQL, RAG y API en el mismo agente	src/pedro/agent.py
R26	Ejecutor con seis herramientas y una corrección	src/pedro/agent.py
R27	Historial por sesión, solo la respuesta pública	src/pedro/memory.py
R28	Chat Streamlit que dibuja datos verificados	app.py
R29	30 casos y seis evaluadores en LangSmith	evals/avaliar_fluxo.py

10. Plan de estudio sugerido

- **Día 1 (Fases 1 y 2):** ejecuta los notebooks 01 y 02. Después lee las fichas R01 a R12 de esta guía.
- **Día 2 (Fases 3 y 4):** ejecuta los notebooks 03 y 04. Lee las fichas R13 a R23.
- **Día 3 (Fase 5):** ejecuta el notebook 05, abre la interfaz y haz una compra en el chat. Lee las fichas R24 a R29.
- **Repaso:** tapa la columna del medio del resumen e intenta explicar cada rúbrica en una frase.